Shirley Zamora
Project 3 (RSA)

Part One:

```
1  #include <iostream>
2  #include <cstdlib>
3  #include <ctime>
```

The program begins with <iostream> which allows the program to print output using cout, and it's then followed by <cstdlib> which provides functions like rand() and srand() to generate random numbers. This is needed because it will help provide random bases for the primality test and provide random primes for the RSA program. <ctime> gives access to time(0): it is able to generate random numbers after each run.

```
long long modexp(long long base, long long exp, long long mod) {
    long long result = 1;
    base %= mod;
```

The first function of the program is Modular Exponentiation:

$$Base^{exp} \bmod mod$$

Using long long, the program is able to handle larger numbers, and the result is always starting at one, the result then stores the final answers after which it reduces the base by the mod.

```
    while (exp > 0) {
        if (exp % 2 == 1)
            result = (result * base) % mod;

        base = (base * base) % mod;
        exp /= 2;
    }
    return result;
}
```

Still in the Modular Exponentiation function, the while loop is looping through the bits of the exponent, if the exponent is odd it will multiply into the result, square the base and then divide the exponent by two (Square and Multiply). This ensures that if the number is big, it won't be multiplying the base repeatedly but instead reducing the exponent after each iteration.

```
bool millerTest(long long d, long long n, long long a) {
    long long x = modexp(a, d, n);
```

This function checks if n is prime, it is computing

$a^d \bmod n$

This function checks if the number behaves like a primer under modular arithmetic

```
    if (x == 1 || x == n - 1)
        return true;

    while (d != n - 1) {
        x = (x * x) % n;
        d *= 2;

        if (x == 1) return false;
        if (x == n - 1) return true;
    }
    return false;
```

If it matches with the expected prime behavior then the bool function will be true, but if it is false the while loop will keep squaring until reaching n-1 until it reaches values that are expected for prime. The millerTest checks if it hits 1 too early (if (x == 1)), this returns false as it's composite.

However if it reaches n-1 the test considers it prime (x == n-1). This is a fast way to test large primes and it's used in RSA key generation .

```
bool isPrime(long long n, int k) {
    if (n < 2) return false;
    if (n % 2 == 0 && n != 2) return false;
```

The isPrime function returns false if the number is less than two or greater than 2.

```
    long long d = n - 1;
    while (d % 2 == 0)
        d /= 2;

    for (int i = 0; i < k; i++) {
        long long a = 2 + rand() % (n - 3);

        if (!millerTest(d, n, a))
            return false;
    }
    return true;
}
```

The function rewrites n-1 to d x 2^r which will be used for the Miller Rabin algorithm. N-1 is divided by 2 until it becomes odd, the function then runs the millerTest (int k) using random values, with each loop it ensures the number is actually a prime number and not a composite.

```
long long gcd(long long a, long long b) {
    while (b != 0) {
        long long r = a % b;
        a = b;
        b = r;
    }
    return a;
}
```

The gcd function implements the Euclidean algorithm by finding the greatest common divisor of two numbers; it works by using a mod b until b is zero. The remaining value of a is the GCD. This makes sure that the public exponent is coprime with phi of n (GCD is one), the system won't work if the GCD is not one.

```
long long egcd(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }

    long long x1, y1;
    long long g = egcd(b, a % b, x1, y1);

    x = y1;
    y = x1 - (a / b) * y1;

    return g;
}
```

Egcd is the extended euclidean algorithm : a*x + b*y = gcd(a,b) it repeats this until b = 0. Then it does back substituting(building it back up) until it is able to have all the information needed for the modular inverses, basically computing the final values of x and y needed to reach the original outputs.

```
long long modInverse(long long a, long long m) {
    long long x, y;
    long long g = egcd(a, m, x, y);

    if (g != 1)
        return -1;

    return (x % m + m) % m;
}
```

modInverse used the extended Euclidean algorithm to compute the inverse of a number.

$$a*x = 1 \bmod m$$

a and m have to be coprime, and if it's not possible -1 is returned, if there is a coprime it makes sure the result is positive. (x % m) if it's negative then (+m) shifts it back into a positive and %m makes sure it's within range of 0 to m-1. If its positive it still remains the same (unchanged).

Outputs:

This is the test for the part one functions that will be needed for RSA:

```
int main() {
    srand(time(0));

    cout << "Miller Rabins Test:\n";
    cout << "29 is " << (isPrime(29, 5) ? "Prime" : "Not Prime") << endl;
    cout << "91 is " << (isPrime(91, 5) ? "Prime" : "Not Prime") << endl;
```

```
Miller Rabins Test:
29 is Prime
91 is Not Prime
```

```
  cout << "GCD Test:\n";
  cout << "gcd(48, 18) = " << gcd(48, 18) << endl;
```

```
GCD Test:
gcd(48, 18) = 6
```

```
    cout << "Mod Inverse Test:\n";
    long long inv = modInverse(3, 11);
```

If the inverse is -1 then the inverse doesn't exist this is implemented with an if statement

```
cout << "Mod Inverse Test:\n";
long long inv = modInverse(3, 11);

if (inv == -1)
    cout << "No inverse exists\n";
else
    cout << "Inverse of 3 mod 11 = " << inv << endl;
```

```
Mod Inverse Test:
Inverse of 3 mod 11 = 4
```

```
  cout << "Square and Multiply Test:\n";
  cout << "2^20 mod 17 = " << modexp(2, 20, 17) << endl;

  return 0;
```

```
Square and Multiply Test:
2^20 mod 17 = 16
```

Part Two (Part A):

```
long long generatePrime(int min, int max) {
    while (true) {
        long long num = min + rand() % (max - min);
        if (num % 2 == 0) num++;

        if (isPrime(num, 5))
            return num;
    }
}
```

generatePrime produces a random prime number within range (min and max) it makes sure that the number is odd and tests it using isPrime. It keeps looping until it passes the primality tests. RSA needs two large prime numbers and this makes sure the prime isn't composite.

```
void generateRSA(long long &n, long long &e, long long &d) {
    long long p = generatePrime(50, 200);
    long long q = generatePrime(50, 200);
```

generateRSA picks primes in the stated range, in this case I chose 50 and 200 to try and keep this program fast.

```
  while (p == q)
      q = generatePrime(50, 200);

  n = p * q;
  long long phi = (p - 1) * (q - 1);

  do {
      e = 2 + rand() % (phi - 2);
  } while (gcd(e, phi) != 1);

  d = modInverse(e, phi);
```

Generate RSA functions takes two primes p and q, and makes sure the values are not the same. If they are, it regenerates. Then the program multiplies p and q and the result is stored in n, which is the modulus. Then it computes Euler's totient which is (p - 1)(q - 1), this counts how many numbers are prime to n which is needed to make the keys.

e = 2 + rand()% (phi -2) chooses a number between the range 2 and phi - 2, this helps avoid picking random values that are 1 and phi. The while loop makes sure that e does not share any factors with phi and makes sure that the modular inverse exists. Then the d = modInverse(e,phi) creates the private key which will be used for decryption.

```
long long encrypt(long long x, long long e, long long n) {
    return modexp(x, e, n);
}
```

This function encrypts :

$$x^e \mod n$$

Using the modexp function, this works as a wrapper function.

```
long long decrypt(long long y, long long d, long long n) {
    return modexp(y, d, n);
}
```

This function decrypts:

$y^d \bmod n$

Using the modexp function, this works as a wrapper function.

```
    long long x = rand() % n;
    cout << "\nPlaintext x = " << x << endl;

    long long y = encrypt(x, e, n);
    cout << "Ciphertext y = " << y << endl;

    long long decrypted = decrypt(y, d, n);
    cout << "Decrypted x = " << decrypted << endl;
```

Long long extends the standards of int and rand generates a message smaller than n, this is so that the message is within the range.

```
Plaintext x = 2916
Ciphertext y = 11527
Decrypted x = 2916
```

```
  cout << "Public Key: " << n << ", " << e << "\n";
  cout << "Private Key: " << d << "\n";
```

```
Public Key: 18857, 8911
Private Key: 271
```

Outputs public key and private key.

```
    cout << "p = " << p << ", q = " << q << endl;
    cout << "n = " << n << ", phi = " << phi << endl;
}
```

Outputs p, q and phi.

```
p = 109, q = 173
n = 18857, phi = 18576
```

```
long long n, e, d;

generateRSA(n, e, d);
```

n is the variable that stores the modulus in RSA (n = p x q), this is used for encryption and decryption and it is part of the public key. The e is used to store the public exponent which is used for gcd(e,phi) =1, and for encryption $y = x^e \bmod n$. And the d is used for the private exponent, it is used as the modular inverse of e ($d = e^{-1} (\text{mod phi})$). And its used for decryption $x = y^d \bmod n$. generateRSA is defined in my code and it takes n, e, and d by reference and stores it into the variables.

```
if (x == decrypted)
    cout << "Match\n";
else
    cout << "Error wrong\n";
```

```
Match
```

If Statement to see if the decrypted message matches the original message.